Testing Fundamentals: What, Why, and How

3 min read Updated 24 Mar 2026 Topic 40 of 62

9.1 – Testing Fundamentals: What, Why, and How

🎯 What Interviewers Actually Ask

Question	Level	Frequency
"What types of tests do you write for React apps?"	🟡	★★★★☆
"What is the testing pyramid?"	🟡	★★★☆☆
"What tools do you use for testing React?"	🟡	★★★★☆
"What is the difference between unit and integration tests?"	🟡	★★★☆☆

The Three Types of Tests

Integration tests test how multiple components work together. They verify that data flows correctly between parents and children, that context providers supply the right values, and that user workflows complete successfully. Example: "Can a user add an item to the cart and see the updated total?"

End-to-end (E2E) tests test the entire application from the user's perspective, running in a real browser. They cover navigation, form submissions, API interactions, and full user flows. Tools: Cypress, Playwright. These are slow and expensive to maintain.

The Testing Pyramid

The recommended ratio is: many unit tests at the base, fewer integration tests in the middle, and very few E2E tests at the top. Most React interview questions focus on unit and integration tests using React Testing Library.

The Testing Stack

Tool	Purpose
Jest	Test runner, assertions, mocking. Comes pre-configured with Create React App
React Testing Library (RTL)	Renders components and provides queries to find elements the way a user wou
@testing-library/user-event	Simulates user interactions more realistically than `fireEvent`.

Service Worker)	Intercepts network requests for testing API interactions without a real backend.
Cypress / Playwright	E2E testing in a real browser.

The React team officially recommends React Testing Library over Enzyme. If an interviewer mentions Enzyme, it's likely a legacy codebase.

The Testing Philosophy

React Testing Library is built on one guiding principle: **test the way users interact with your app, not implementation details.**

This means:

- Query elements by **role, label, placeholder, or text**, not by class names or component internals.
- Test **behavior** ("when I click the button, the count increases"), not **state** ("the count state variable equals 1").
- Don't test whether a specific hook was called or whether a state variable changed. Test what the user sees and experiences.

jsx

JSX Copy

```jsx
// ❌ Testing implementation details
expect(component.state.count).toBe(1);
expect(useState).toHaveBeenCalled();
```

Interview Answer

"I write three types of tests for React apps. Unit tests for individual components and hooks using Jest and React Testing Library. Integration tests that verify user workflows across multiple components. And a few E2E tests with Cypress for critical user flows like checkout or signup. I follow the React Testing Library philosophy of testing user behavior, not implementation details. I query elements by role and text, not by class names or internal state."

QUICK CHECK — no marks, just for you.

1. According to the testing pyramid described in the lesson, what is the recommended ratio of test types in a React app?

- ◯ Mostly integration tests, with unit and E2E tests used only for edge cases
- ◯ Many E2E tests at the base, fewer integration tests in the middle, very few unit tests at the top
- ◯ Many unit tests at the base, fewer integration tests in the middle, very few E2E tests at the top
- ◯ An equal number of unit, integration, and E2E tests

Check answer

2. In the lesson's testing stack, which tool is responsible for intercepting network requests so you can test API interactions without a real backend?

- @testing-library/user-event
- Jest, using its built-in test runner and assertions
- React Testing Library, via its element queries

Check answer

3. A teammate writes a test asserting `expect(useState).toHaveBeenCalled()` for a counter component. Based on the React Testing Library philosophy in the lesson, why is this a poor test?

- Hooks can never be asserted on inside a test file, so the assertion will always throw
- It is an integration test, and counters should only ever be covered by unit tests
- It should be rewritten as an E2E test in Cypress, since hooks require a real browser
- It tests an implementation detail rather than the behavior the user actually observes

Check answer

‹ Previous
Debouncing & Throttling

Next ›
React Testing Library: Practical Guide

LEARN MORE

ARTICLES & DOCS

Jest — official docs
jestjs.io

VIDEOS



React testing for beginners — start here
YouTube



Intro to React testing (Jest + RTL)
YouTube



React unit testing crash course
YouTube

Hand-picked references for this lesson.